available()

Since we have already looked at sending data to our computer a few chapters ago, we will first talk about reading data and get back to sending it later in this chapter. Before we can read data from the serial line, we need to know if data is available or not. That's where the appropriately named `available()` function comes in.

```
Serial.available()
```

This function has no parameters but will instead return the number of available bytes waiting in the serial buffer. Remember, the hardware serial port on the Arduino microcontroller has a buffer that can store up to 128 bytes of information so that it doesn't get lost. If no data is waiting for us, it will return 0 and we can use this in an `if` statement like we did in our project code, as follows:

```
if (Serial.available()) {
```

If no data is available in the buffer, then this function will return 0 and evaluate as `false`. On the other hand, if any data is available, the function will return a value other than 0 that will evaluate as `true` and we can proceed to read from the buffer. In our example, it didn't matter how much data is in the buffer, only that there was data to be read.

read()

Now that we know there is data waiting for us in our buffer, we can use the `read()` function to do something with it. While the syntax is simple enough, there are some unusual characteristics that we will need to work through before moving on.

```
Serial.read()
```

This function simply returns the first byte of information available in the serial buffer. Because the way our serial communications are structured, each character that we send to the Arduino through the Serial Monitor will be converted to that character's ASCII character value. See, I warned you this would come up again. For example, if we were to send the Arduino the number `1`, instead of receiving the numerical integer `1`, the Arduino will actually receive the numerical value `49` corresponding to that character's ASCII character code. I know, this is not the most intuitive process here but bear with me.

■ **Note** As we continue our discussion of characters, remember that the use of single and double quotes is not interchangeable. Single quotes are used for characters, as in the character 'a', while double quotes are used for strings of text, as in "Hello, world!"

When we read a byte from the serial line, we are reading one of the ASCII values from 0 to 127 that correspond to the ASCII characters, shown in Table 10-1, that are received through the serial hardware one byte at a time. We can either leave the numerical value alone by assigning it to a numerical data type, or we could store it as a character by using the `char` data type. From our example code, it is as follows:

```
byte incomingByte = Serial.read();
```